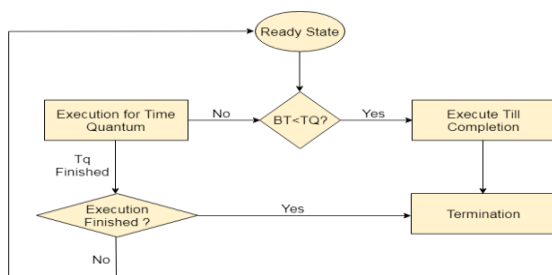


OS

1.RR

Round Robin scheduling algorithm is one of the most popular scheduling algorithm which can actually be implemented in most of the operating systems. This is the **preemptive version** of first come first serve scheduling. The Algorithm focuses on Time Sharing. In this algorithm, every process gets executed in a **cyclic way**. A certain time slice is defined in the system which is called time **quantum**. Each process present in the ready queue is assigned the CPU for that time quantum, if the execution of the process is completed during that time then the process will **terminate** else the process will go back to the **ready queue** and waits for the next turn to complete the execution.



Advantages

1. It can be actually implementable in the system because it is not depending on the burst time.
2. It doesn't suffer from the problem of starvation or convoy effect.
3. All the jobs get a fare allocation of CPU.

Disadvantages

1. The higher the time quantum, the higher the response time in the system.
2. The lower the time quantum, the higher the context switching overhead in the system.
3. Deciding a perfect time quantum is really a very difficult task in the system.

R Scheduling Example

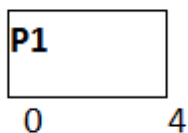
In the following example, there are six processes named as P1, P2, P3, P4, P5 and P6. Their arrival time and burst time are given below in the table. The time quantum of the system is 4 units. According to the algorithm, we have to maintain the ready queue and the Gantt chart. The structure of both the data structures will be changed after every scheduling.

Ready Queue: Initially, at time 0, process P1 arrives which will be scheduled for the time slice 4 units. Hence in the ready queue, there will be only one process P1 at starting with CPU burst time 5 units.

Process ID	Arrival Time	Burst Time
1	0	5
2	1	6
3	2	3
4	3	1
5	4	5
6	6	4
P1		
5		

GANTT chart

The P1 will be executed for 4 units first.



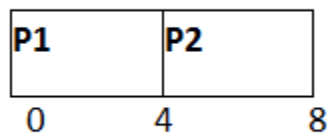
Ready Queue

Meanwhile the execution of P1, four more processes P2, P3, P4 and P5 arrives in the ready queue. P1 has not completed yet, it needs another 1 unit of time hence it will also be added back to the ready queue.

P2	P3	P4	P5	P1
6	3	1	5	1

GANTT chart

After P1, P2 will be executed for 4 units of time which is shown in the Gantt chart.



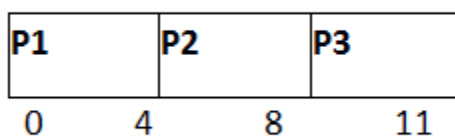
Ready Queue

During the execution of P2, one more process P6 is arrived in the ready queue. Since P2 has not completed yet hence, P2 will also be added back to the ready queue with the remaining burst time 2 units.

P3	P4	P5	P1	P6	P2
3	1	5	1	4	2

GANTT chart

After P1 and P2, P3 will get executed for 3 units of time since its CPU burst time is only 3 seconds.



Ready Queue

Since P3 has been completed, hence it will be terminated and not be added to the ready queue. The next process will be executed is P4.

P4	P5	P1	P6	P2
1	5	1	4	2

GANTT chart

After, P1, P2 and P3, P4 will get executed. Its burst time is only 1 unit which is lesser then the time quantum hence it will be completed.

P1	P2	P3	P4	
0	4	8	11	12

Ready Queue

The next process in the ready queue is P5 with 5 units of burst time. Since P4 is completed hence it will not be added back to the queue.

P5	P1	P6	P2
5	1	4	2

GANTT chart

P5 will be executed for the whole time slice because it requires 5 units of burst time which is higher than the time slice.

P1	P2	P3	P4	P5	
0	4	8	11	12	16

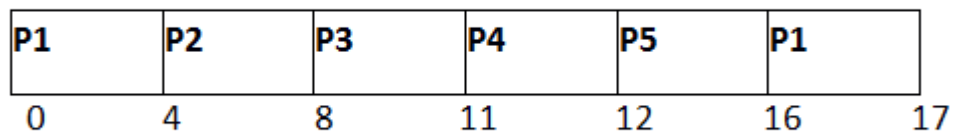
Ready Queue

P5 has not been completed yet; it will be added back to the queue with the remaining burst time of 1 unit.

P1	P6	P2	P5
1	4	2	1

GANTT Chart

The process P1 will be given the next turn to complete its execution. Since it only requires 1 unit of burst time hence it will be completed.



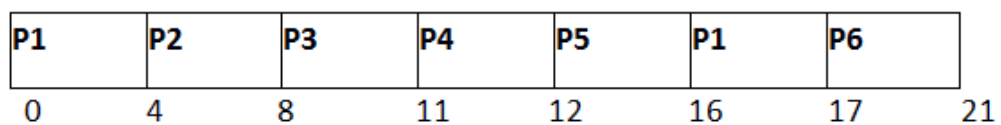
Ready Queue

P1 is completed and will not be added back to the ready queue. The next process P6 requires only 4 units of burst time and it will be executed next.

P6	P2	P5
4	2	1

GANTT chart

P6 will be executed for 4 units of time till completion.



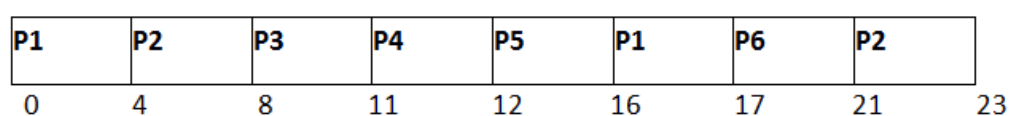
Ready Queue

Since P6 is completed, hence it will not be added again to the queue. There are only two processes present in the ready queue. The Next process P2 requires only 2 units of time.

P2	P5
2	1

GANTT Chart

P2 will get executed again, since it only requires only 2 units of time hence this will be completed.



Ready Queue

Now, the only available process in the queue is P5 which requires 1 unit of burst time. Since the time slice is of 4 units hence it will be completed in the next burst.

P5
1

GANTT chart

P5 will get executed till completion.

P1	P2	P3	P4	P5	P1	P6	P2	P5	
0	4	8	11	12	16	17	21	23	24

The completion time, Turnaround time and waiting time will be calculated as shown in the table below.

As, we know,

1. Turn Around Time = Completion Time - Arrival Time
2. Waiting Time = Turn Around Time - Burst Time

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	0	5	17	17	12
2	1	6	23	22	16
3	2	3	11	9	6
4	3	1	12	9	8
5	4	5	24	20	15
6	6	4	21	15	11

$$\text{Avg Waiting Time} = (12+16+6+8+15+11)/6 = 76/6 \text{ units}$$

2 Priority Scheduling ?

The Operating System has a major function of deciding the order in which processes / tasks will have access to the processor and the amount of processing time each of these processes or tasks will have. This function of the operating system is termed as process scheduling.

Under process scheduling, the tasks that the operating system performs are:

- Keeping track of the status of the processes
- Allocation of processor to processes
- De-allocation of processor to processes.

All these processes are stored by the operating system in a process table along with an ID allotted to every process (PID), to help keep track of all of them. And, to keep track of their current state the operating system uses a *Process Control Block (PCB)*. The information is updated in the control block as and when the state of the process changes.

But did you wonder what is the criteria that the Operating System uses for the scheduling of these processes? Which process must be allocated with CPU resources first? Well, there are multiple ways in which the Operating System takes these decisions such as execution of the process that takes the least time first, or execution according to the order in which they requested access to the processor and so on. One such important criteria is the priority of the process.

In priority scheduling in OS, processes are executed on the basis of their priority. The jobs / processes with higher priority are executed first. Naturally, you might want to know how priority of processes is decided.

Priority of processes depends some factors such as:

- Time limit
- Memory requirements of the process
- Ratio of average I/O to average CPU burst time

There can be more factors on the basis of which priority of a process / job is determined. This priority is assigned to the processes by the scheduler.

These priorities of processes are represented as simple integers in a fixed range such as 0 to 7, or maybe 0 to 4095. These numbers depend on the type of systems.

Now that we know that each job is assigned a priority on the basis of which it is scheduled, we can move ahead with it's working and types.

Types of Priority Scheduling algorithm

There are two types of priority scheduling algorithm in OS:

Non-Preemptive Scheduling

In this type of scheduling:

- If during the execution of a process, another process with a higher priority arrives for execution, even then the currently executing process will not be disturbed.

- The newly arrived high priority process will be put in next for execution since it has higher priority than the processes that are in waiting state for execution.
- All the other processes will remain in the waiting queue to be processes. Once the execution of the current process is done, the high priority process will be given the CPU for execution.

Preemptive Scheduling

Preemptive Scheduling as opposed to non-preemptive scheduling will preempt (stop and store the currently executing process) the currently running process if a higher priority process enters the waiting state for execution and will execute the higher priority process first and then resume executing the previous process.

Characteristics of Priority Scheduling Algorithm:

- It is a scheduling algorithm that schedules the incoming processes on the basis of the priority.
- Operating systems use it for performing batch processes
- If there exist two jobs / processes in the ready state (ready for execution) that have the same priority, then priority scheduling executed the processes on first come first serve basis. For every job that exists, we have a priority number assigned to it that indicates its priority level.
- If the integer value of the priority number is low, it means that the process has higher priority. (low number = high priority).

Example of Priority Scheduling algorithm:

You've now understood the basics of priority scheduling in os. Let's better your understanding with an example of preemptive scheduling.

Consider five processes from P1 to P5, with their priority, burst time, and arrival time. If you aren't familiar with the terms burst and arrival time, please Google them! We're going to represent these values associated with these processes in a table.

Process	Priority	Burst Time
P1	1	4
P2	2	3
P3	1	7
P4	3	4
P5	2	2

Let's apply whatever we learnt about priority scheduling algorithm. Which process do you think will be executed first? Remember, lower the number, higher the priority. We'll use a gantt chart to keep track of the time, and to check if a process must be preempted.

Time

Process

Of course, the processes that arrive first i.e. at time = 0 will be considered for execution first, which are P1 and P2. The priority of process P1 is higher than P2, and hence it will be executed first.

Time

0

Process

P1

At time = 1, we do not have any new processes arriving, and process P1 continues to execute, and this goes on till time = 4 and that's when the process has finished its execution.

Time

0

1

2

Process

P1

P1

P1

All this while, process P2 was in the waiting queue. Once the execution of process P1 is complete, we pick the next process in the waiting queue according to priority regardless of the arrival time.

However, there's just one process in the waiting queue, so we start the execution of process P2 which has a burst time of 3.

Time

0

1

2

3

Process

P1

P1

P1

P1

At time = 5, process P2 is still executing and the burst time remaining is 2. At time = 6, we see that another process arrives in the waiting queue. The first thing that we need to check is the priority and since the priority of process P3 is greater than P2 we must preempt the current process and start the execution of P3, keeping P2 in ready queue with a remaining burst time of 1 (it already executed for 2 seconds).

Process

Priority

Burst Time

P1

1

4

P2

2

3 1

P3

1

7

P4

3

4

P5

2

2

Time

0

1

2

3

4

5

6

Process

P1

P1

P1

P1

P1

P2

P2

The process P3 has a burst time of 7, and will obviously continue to execute till another process with a higher priority comes in. At time = 11, another process P4 comes in, but has a priority less than that of P3 and hence P3 will continue to execute with 2 units remaining.

Time	0	1	2	3	4	5	6	7	8
Process	P1	P1	P1	P1	P1	P2	P2	P2	P2

P3 will execute till time = 13. We now go to our waiting queue to check for processes. Remember that we pick processes with higher priority **that arrived first**. In our waiting queue, we have processes P2, P4 and P5. We see that P2 and P5 have equal priorities but since the arrival time of P2 is before that of process P5, we must execute process P2 first.

Time	0	1	2	3	4	5	6	7	8	9
Process	P1	P1	P1	P1	P1	P2	P2	P2	P2	P2

P2 just has one unit time of processing left as we had preempted the process to run P3. At time = 14 P2 process completes its execution. Again, we look for a process from the waiting queue that can be executed. In the waiting queue we have P4 and P5. Since P5 has the higher priority, we start its execution.

Time	0	1	2	3	4	5	6	7	8	9
Process	P1	P1	P1	P1	P1	P2	P2	P2	P2	P2

The burst time of process P5 is 2, and hence it completes its execution at time = 16. Now, the only remaining process is process P4, and at time = 16 it starts its execution. Since the burst time of process P4 is 4, and there are no more processes in the waiting queue, and no new processes with higher priority have arrived, P4 continues and completes its execution by time = 20.

Time	0	1	2	3	4	5	6	7	8	9
Process	P1	P1	P1	P1	P1	P2	P2	P2	P2	P2

You have now understood the working of preemptive priority scheduling in os! Moving on, let's take a look at its advantages and disadvantages.

Advantages of Priority Scheduling Algorithm

- High priority processes do not have to wait for their chance to be executed due to the current running process.
- We are able to define the relative importance / priority of processes.
- The applications in which the requirements of time and resources fluctuate are useful.

Disadvantages of Priority Scheduling Algorithm

- Since we only execute high priority processes, this can lead to starvation of the processes that have a low priority. Starvation is the phenomenon in which a process gets infinitely postponed because the resources that are required by the process are never allocated to it, since other processes are executed before it. You can research more about starvation on Google.
- If the system eventually crashes, all of the processes that have low priority will get lost since they are stored in the RAM.

Static and Dynamic Priority

When we were learning about priorities being given to processes, did you ponder about how the priorities were assigned to the process in the first place? Or rather, when was the priority assigned to the process?

In priority based scheduling, we saw that it could be implemented in two ways - preemptive and non-preemptive with the most common implementation being preemptive priority based scheduling. The same way, it can be categorized again on the basis of the method by which the priorities to processes are assigned.

The two classifications are:

- *Static priority*
- *Dynamic priority*

The Static Priority algorithm, assigns priorities to processes at design time, and these assigned priorities do not change, they remain constant for the lifetime of that process. However in the dynamic priority algorithm, the priorities are assigned to the processes at run time and this assignment is based on the execution parameters of the processes such as upcoming deadlines.

Of course, the static priority algorithms are simpler than the dynamic priority algorithms. There are other scheduling algorithms, like multilevel queues and multilevel feedback queues scheduling which are important for a system that provides better performance. Let's take a quick look into them.

Multilevel Queues

When we have multiple processes in the ready queue, there are times when there are similar processes in the waiting queue. Similar how? For example, the processes could be interactive or batch processes, and due to their divergent response time requirements, they would have dissimilar scheduling needs. It makes sense if these similar processes are classified into groups.

This is called multilevel queue scheduling. We have different queues for different processes that have different scheduling requirements. For example, for system processes, we can have a separate queue which schedules them according to FCFS method. Interactive processes will have their queue and could be scheduled using SJF scheduling method and so on. These queues have their own priorities as well, and hence known as multilevel queue scheduling. queue schedu

3. Multilevel Feedback Queues

Multilevel feedback queues are just like the multilevel queues, with one slight difference. If a process is part of Queue 1 in multilevel queue scheduling, then the process cannot switch and move into another queue. But here, it can. This scheduling continues to analyze the time of execution of processes and makes the switch between processes in different queues accordingly, hence the name feedback queues.

Multilevel Feedback Queues

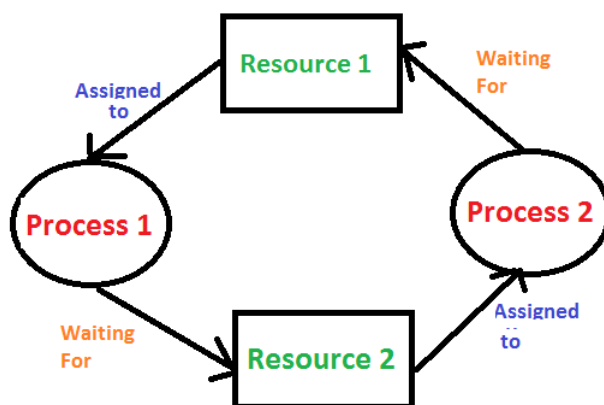
Multilevel feedback queues are just like the multilevel queues, with one slight difference. If a process is part of Queue 1 in multilevel queue scheduling, then the process cannot switch and move into another queue. But here, it can. This scheduling continues to analyze the time of execution of processes and makes the switch between processes in different queues accordingly, hence the name feedback queues.

In the previous post, we have discussed [Deadlock Prevention and Avoidance](#). In this post, the Deadlock Detection and Recovery technique to handle deadlock is discussed.

4. Deadlock Detection :

1. If resources have a single instance –

In this case for Deadlock detection, we can run an algorithm to check for the cycle in the Resource Allocation Graph. The presence of a cycle in the graph is a sufficient condition for deadlock.



2. In the above diagram, resource 1 and resource 2 have single instances. There is a cycle $R1 \rightarrow P1 \rightarrow R2 \rightarrow P2$. So, Deadlock is Confirmed.

3. If there are multiple instances of resources –

Detection of the cycle is necessary but not sufficient condition for deadlock detection, in this case, the system may or may not be in deadlock varies according to different situations.

Deadlock Recovery :

A traditional operating system such as Windows doesn't deal with deadlock recovery as it is a time and space-consuming process. Real-time operating systems use Deadlock recovery.

1. Killing the process –

Killing all the processes involved in the deadlock. Killing process one by one. After killing each process check for deadlock again keep repeating the process till the system recovers from deadlock. Killing all the processes one by one helps a system to break circular wait condition.

2. Resource Preemption –

Resources are preempted from the processes involved in the deadlock, preempted

resources are allocated to other processes so that there is a possibility of recovering the system from deadlock. In this case, the system goes into starvation

5. Page Replacement Algorithms in Operating Systems

In an operating system that uses paging for memory management, a page replacement algorithm is needed to decide which page needs to be replaced when a new page comes in.

Page Fault: A page fault happens when a running program accesses a memory page that is mapped into the virtual address space but not loaded in physical memory. Since actual physical memory is much smaller than virtual memory, page faults happen. In case of a page fault, Operating System might have to replace one of the existing pages with the newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce the number of page faults.

Page Replacement Algorithms:

1. First In First Out (FIFO): This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames. Find the number of page faults.

Page reference		1, 3, 0, 3, 5, 6, 3						
1	3	0	3	5	6	3		
		0	0	0	0	3		
	3	3	3	3	6	6		
1	1	1	1	5	5	5		
Miss	Miss	Miss	Hit	Miss	Miss	Miss		
Total Page Fault = 6								

Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots → **3 Page Faults**.

when 3 comes, it is already in memory so → **0 Page Faults**. Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. → **1 Page Fault**. 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 → **1 Page Fault**. Finally, when 3 come it is not available so it replaces 0 **1 page fault**.

[Belady's anomaly](#) proves that it is possible to have more page faults when increasing the number of page frames while using the First in First Out (FIFO) page replacement algorithm. For example, if we consider reference strings 3, 2, 1, 0, 3, 2, 4, 3, 2, 1, 0, 4, and 3 slots, we get 9 total page faults, but if we increase slots to 4, we get 10-page faults.

2. Optimal Page replacement: In this algorithm, pages are replaced which would not be used for the longest duration of time in the future.

Example-2: Consider the page references 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frame. Find number of page fault.

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots → **4 Page faults**
 0 is already there so → **0 Page fault**. when 3 came it will take the place of 7 because it is not used for the longest duration of time in the future. → **1 Page fault**. 0 is already there so → **0 Page fault**. 4 will takes place of 1 → **1 Page Fault**.

Now for the further page reference string → **0 Page fault** because they are already available in the memory.

Optimal page replacement is perfect, but not possible in practice as the operating system cannot know future requests. The use of Optimal Page replacement is to set up a benchmark so that other replacement algorithms can be analyzed against it.

3. Least Recently Used: In this algorithm, page will be replaced which is least recently used.

Example-3: Consider the page reference string 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 with 4 page frames. Find number of page faults.

Page reference	7,0,1,2,0,3,0,4,2,3,0,3,2,3														No. of Page frame - 4
7	0	1	2	0	3	0	4	2	3	0	3	2	3		
			2	2	2	2	2	2	2	2	2	2	2		
		1	1	1	1	1	4	4	4	4	4	4	4		
	0	0	0	0	0	0	0	0	0	0	0	0	0		
7	7	7	7	7	3	3	3	3	3	3	3	3	3		
Miss	Miss	Miss	Miss	Hit	Miss	Hit	Miss	Hit	Hit	Hit	Hit	Hit	Hit		

Total Page Fault = 6

Here LRU has same number of page fault as optimal but it may differ according to question.

Initially, all slots are empty, so when 7 0 1 2 are allocated to the empty slots → **4 Page faults**
 0 is already their so → **0 Page fault**. when 3 came it will take the place of 7 because it is least recently used → **1 Page fault**

0 is already in memory so → **0 Page fault**.

4 will takes place of 1 → **1 Page Fault**

Now for the further page reference string → **0 Page fault** because they are already available in the memory.

4. Most Recently Used (MRU): In this algorithm, page will be replaced which has been used recently. Belady's anomaly can occur in this algorithm.

GATE CS Corner Questions

Practicing the following questions will help you test your knowledge. All questions have been asked in GATE in previous years or in GATE Mock Tests. It is highly recommended that you practice them.

6 Protection and Security

Protection and security requires that computer resources such as CPU, softwares, memory etc. are protected. This extends to the operating system as well as the data in the system. This can be done by ensuring integrity, confidentiality and availability in the operating system. The system must be protect against unauthorized access, viruses, worms etc.

Threats to Protection and Security

A threat is a program that is malicious in nature and leads to harmful effects for the system. Some of the common threats that occur in a system are –

Virus

Viruses are generally small snippets of code embedded in a system. They are very dangerous and can corrupt files, destroy data, crash systems etc. They can also spread further by replicating themselves as required.

Trojan Horse

A trojan horse can secretly access the login details of a system. Then a malicious user can use these to enter the system as a harmless being and wreak havoc.

Trap Door

A trap door is a security breach that may be present in a system without the knowledge of the users. It can be exploited to harm the data or files in a system by malicious people.

Worm

A worm can destroy a system by using its resources to extreme levels. It can generate multiple copies which claim all the resources and don't allow any other processes to access them. A worm can shut down a whole network in this way.

Denial of Service

These type of attacks do not allow the legitimate users to access a system. It overwhelms the system with requests so it is overwhelmed and cannot work properly for other user.

Protection and Security Methods

The different methods that may provide protect and security for different computer systems are –

Authentication

This deals with identifying each user in the system and making sure they are who they claim to be. The operating system makes sure that all the users are authenticated before they access the system. The different ways to make sure that the users are authentic are:

- **Username/ Password**

Each user has a distinct username and password combination and they need to enter it correctly before they can access the system.

- **User Key/ User Card**

The users need to punch a card into the card slot or use their individual key on a keypad to access the system.

- **User Attribute Identification**

Different user attribute identifications that can be used are fingerprint, eye retina etc. These are unique for each user and are compared with the existing samples in the database. The user can only access the system if there is a match.

One Time Password

These passwords provide a lot of security for authentication purposes. A one time password can be generated exclusively for a login every time a user wants to enter the system. It cannot be used more than once. The various ways a one time password can be implemented are –

- **Random Numbers**

The system can ask for numbers that correspond to alphabets that are pre arranged. This combination can be changed each time a login is required.

- **Secret Key**

A hardware device can create a secret key related to the user id for login. This key can change each time.

This kind of scheduling is much more flexible than the basic multilevel queue scheduling, and they also reduce the response time. However, the processes in lower priority queues might suffer from starvation in this scheduling method.

Conclusion

- Priority based scheduling in operating systems is the scheduling of processes based on their priority.
 - This priority depends on factors such as time limit, memory requirements of the process and ratio of average I/O to average CPU burst time.
- There are mainly two types of priority scheduling -- non-preemptive (where another process can only be executed once the current process is done executing) and preemptive (where a higher priority process pauses the current running process and completes its execution first).
- The main disadvantage of priority scheduling is that lower priority processes can suffer from starvation.
- Priorities of processes can be decided in two ways - static or dynamic.
- There are more types of scheduling such as multilevel queues and multilevel feedback queues which essentially group similar processes and have separate queues for them so that they can follow different scheduling algorithms.